

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade
Anand Nagar, Krishnankoil, Srivilliputtur, Virudhunagar (Dt.) - 626126 | OMR, Chennai, Tamil Nadu
info@kalasalingam.ac.in | www.kalasalingam.ac.in



Social Media and Script Idea Generator

Project ID: PRJ105

Team ID: T105

Team GitHub Repository: https://github.com/RishithaKamepalli/Social-Media-and-Script-idea-Generator-gen-AI

Student Name	Registration Number	Individual GitHub Link
Kamepalli swathi Rishitha	99230040321	https://github.com/RishithaKamepalli/Social-Media-and-Script-idea-Generator-gen-AI
Marripudi Kavya Sri	99230041287	https://github.com/kavya-26331/Social-and-Script-Idea-Generator
Shaik Suhana	99230040165	https://github.com/ShaikSuhana11/Social-and-Script-Idea-Generator
Sure Veeramani	99230040173	https://github.com/veeramanisure3-art/Social-and-Script-Idea-Generator-
Allipuram Venkata Naga Sai Charan	99230040254	https://github.com/Charanyadav1328/Social-and-Script-Idea-Generator

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ..... Section 1

2. Problem Statement & Objectives ..... Section 2

3. System Architecture (RAG Pipeline) ..... Section 3

4. Implementation Details & Tech Stack ..... Section 4

5. Results and Testing ..... Section 5

6. Conclusion & Future Scope ..... Section 6

1. Introduction

The rapid growth of social media platforms has increased the need for creative, attractive, and engaging digital content. Content creators, businesses, and digital marketers must regularly develop new post ideas, captions, hashtags, and video scripts to connect with their target audiences. However, creating fresh and platform-specific content can be time-consuming and requires continuous creativity. Maintaining a consistent brand identity and adapting content to different audiences can also be challenging when using traditional content creation methods.

The AI-Powered Social Media Content and Script Idea Generator Using RAG and Groq API is a web-based application designed to make the content creation process easier and more efficient. It allows users to enter a topic, select a social media platform, specify a target audience, and choose a preferred writing tone. Based on these inputs, the system generates creative post ideas, captions, relevant hashtags, Instagram Reel scripts, and YouTube video scripts. Users can also upload reference documents, such as brand guidelines, product details, and previously created scripts, to help generate content that matches their requirements.

The application uses Retrieval-Augmented Generation (RAG) to retrieve relevant information from uploaded documents and provide useful context for content generation. PyPDF extracts text from PDF files, Hugging Face Sentence Transformers converts text into embeddings, and ChromaDB stores the embeddings for semantic search. When a user requests content, the system retrieves relevant information and combines it with the user's instructions. The Groq API provides access to a Large Language Model (LLM), which generates personalized and context-aware content. Python and Streamlit are used to develop the application and provide an interactive user interface, making social media content creation faster, simpler, and more convenient.

2. Problem Statement & Objectives

2.1 Problem Statement

In today's digital world, social media platforms such as Instagram, YouTube and Facebook play an important role in marketing, communication and content sharing. Content creators, small business owners and digital marketers often face difficulties in generating creative post ideas, writing engaging captions, selecting suitable hashtags and preparing video scripts. Creating quality content manually requires time, creativity and a proper understanding of the target audience. Existing AI content generators may produce generic responses and may not consider specific brand guidelines, reference documents or previously created content. Therefore, an intelligent and accessible system is required to automate social media content creation by providing personalized, platform-specific and context-aware content using Generative AI and Retrieval-Augmented Generation (RAG).

2.2 Objectives

- Develop an AI-powered web application for generating social media content and video scripts.
- Generate creative post ideas, captions and relevant hashtags based on user inputs.
- Provide separate content generation options for platforms such as Instagram, YouTube and Facebook.
- Allow users to customize content based on topic, target audience, platform and writing tone.
- Implement Retrieval-Augmented Generation (RAG) to retrieve relevant information from uploaded reference documents.
- Integrate Hugging Face Sentence Transformers for generating text embeddings.
- Use ChromaDB to store and retrieve relevant document information through semantic search.
- Integrate the Groq API with an LLM to generate personalized and context-aware content.
- Provide options to view, edit and download generated content.

2.3 Scope of the Project

The scope of this project is to develop an AI-powered web application that simplifies social media content creation using Generative AI and RAG. Users can enter topics, platforms, target audiences, content types, and tones, and upload reference documents. The system processes these documents using PyPDF, Hugging Face, and ChromaDB to retrieve relevant information. The retrieved context is then sent to an LLM through the Groq API to generate personalized captions, hashtags, post ideas, and video scripts. Streamlit provides a simple interface for generating, viewing, editing, and downloading the content. The main goal is to save time and improve the relevance, creativity, and consistency of social media content.

3. System Architecture (RAG Pipeline)

The proposed system is designed as an AI-based web application for generating social media content. Streamlit provides the user interface, while the RAG pipeline handles document processing and information retrieval. Groq API is used to access the Large Language Model for content generation.

Users can provide content requirements such as topic, platform, target audience, content type, and tone. They can also upload reference documents such as brand guidelines, product information, and previous scripts. The uploaded content is processed, converted into embeddings, stored in ChromaDB, and retrieved when relevant information is required.

3.1 Main System Components

Component	Responsibility
Streamlit Interface	Provides the user interface for entering requirements, uploading documents, viewing and downloading generated content.
User Input Module	Collects topic, platform, audience, tone and content type from the user.
Document Processing Module	Extracts text from uploaded PDF documents using PyPDF and divides it into smaller chunks.
Embedding Module	Converts document chunks into numerical embeddings using Hugging Face Sentence Transformers.
ChromaDB	Stores document embeddings and retrieves relevant information based on semantic similarity.
RAG Module	Connects user queries with relevant information retrieved from the documents.
Groq AI Service	Sends the prompt and retrieved context to the LLM and receives the generated response.
Output Module	Displays generated ideas, captions, hashtags and scripts and provides download options.

3.2 Overall Application Workflow

- User Input & Document Upload – User enters topic, platform, audience, tone, content type, and uploads reference documents.
- Document Processing – PyPDF extracts text from uploaded documents and divides it into smaller chunks.
- Embedding & Retrieval – Hugging Face Sentence Transformers creates embeddings, which are stored in ChromaDB for relevant information retrieval.
- RAG + Groq AI Generation – Retrieved context is combined with user inputs and sent through the Groq API to the LLM for content generation.
- Content Output – The generated captions, hashtags, post ideas, or scripts are displayed in Streamlit for viewing, editing, and downloading.The browser speaks the question. After speech playback, the application starts speech recognition and captures the candidate's response as text.

3.3 Document Processing and Embedding

The document-processing stage prepares uploaded information for retrieval.

First, PyPDF extracts text from PDF files. Large amounts of extracted text are divided into smaller chunks so that relevant sections can be retrieved efficiently.

Each chunk is then converted into a numerical vector using a Hugging Face Sentence Transformer model. These vectors represent the semantic meaning of the text and are stored in ChromaDB along with the original text.

When the user enters a topic, the system compares the query with the stored embeddings and identifies the most relevant document sections.

3.4 RAG-Based Content Generation

he RAG process consists of two major stages:

Retrieval:

The system searches ChromaDB and retrieves document chunks that are relevant to the user's request.

Generation:

The retrieved context is combined with the user's requirements and sent to the LLM through the Groq API.

For example, if a user asks for an Instagram Reel about a particular product, the system can retrieve relevant product information from the uploaded documents and use it while generating the Reel script.

This helps the generated content remain consistent with the provided reference material.

3.5 Groq API and Content Generation

The Groq API is used to communicate with the selected Large Language Model.
The prompt contains:

- User's topic.
- Selected platform.
- Target audience.
- Desired tone.
- Content type.
- Retrieved document context.

The LLM processes this information and generates the requested content, such as:

- Social media post ideas.
- Captions.
- Hashtags.
- Instagram Reel scripts.
- YouTube video scripts.

The generated response is then returned to the Streamlit interface.

3.6 Architecture Flow – Text Representation

User
↓
Streamlit Interface
↓
Topic + Platform + Audience + Tone + Content Type
↓
Document Upload
↓
PyPDF
↓
Text Extraction
↓
Text Chunking
↓
Hugging Face Sentence Transformers
↓
Embeddings
↓
ChromaDB
↓
RAG Retrieval
↓
Relevant Document Context
↓
User Input + Retrieved Context
↓
Prompt
↓
Groq API
↓
LLM
↓
Generated Social Content
↓
Streamlit Output
↓
View / Edit / Download

4.Implementation Details & Tech Stack

4.1 Technology Stack

Component	Technology Used	Purpose in the Project
Programming Language	Python	Implements the application logic, document processing and AI integration.
Frontend Framework	Streamlit	Provides an interactive interface for user inputs, document uploads and generated results.
LLM Inference	Groq API	Connects the application with Large Language Models for content generation.
Generative AI	Large Language Models (LLMs)	Generates creative ideas, captions, hashtags and video scripts.
RAG Framework	LangChain	Coordinates document retrieval and communication with the language model.
Embedding Model	Hugging Face Sentence Transformers	Converts document text into numerical vector representations.
Vector Database	ChromaDB	Stores document embeddings and retrieves relevant information through similarity search.
PDF Processing	PyPDF	Extracts textual information from uploaded PDF documents.
Backend Framework	FastAPI	Can provide separate API endpoints for backend operations.
Database	PostgreSQL	Can store user information, generated content and content history.

4.2 Important Modules

A. User Input and Interface Module

Uses Streamlit to collect the topic, platform, target audience, tone, and content type from the user.

B. Document Processing Module

Uploads reference PDF files, extracts text using PyPDF, and divides the text into smaller chunks.

C. Embedding and Vector Storage Module

Uses Hugging Face Sentence Transformers to generate embeddings and stores them in ChromaDB for retrieval.

D. RAG Module

Retrieves relevant information from stored documents and combines it with the user's input to provide context to the LLM.

E. AI Content Generation Module

Uses Groq API and an LLM to generate content ideas, captions, hashtags, and video scripts.

F. Output Module

Displays the generated content through Streamlit and allows users to review and use the results.

4.3 RAG Pipeline Functions and Data Structures

Function / Interface	Role
extract_pdf_text(file)	Extracts readable text from uploaded PDF documents using PyPDF.
split_text(text)	Divides extracted document text into smaller chunks.
generate_embeddings(chunks)	Converts text chunks into numerical vectors using Sentence Transformers.
store_embeddings(chunks)	Stores document chunks and their embeddings in ChromaDB.
retrieve_context(query)	Searches the vector database and retrieves relevant document information.
create_prompt(input, context)	Combines user requirements with retrieved information to construct the LLM prompt.
generate_content(prompt)	Sends the prompt to the Groq API and obtains the generated response.

4.4 Interview Evaluation Output

The system generates social media content based on the user's input and uploaded documents.
Types of Output:

- Content Ideas
- Social Media Captions
- Relevant Hashtags
- Instagram Reel Scripts
- YouTube Video Scripts

The generated content is displayed through the Streamlit interface for users to review and use.

5. Results and Testing

5.1 Implemented Results

The application provides a user-friendly Streamlit interface for generating AI-powered social media content. Users can enter their requirements, select content preferences, and generate suggestions using AI.

5.1.1. User Input Module

The application allows users to enter the topic or niche and specify their target audience.

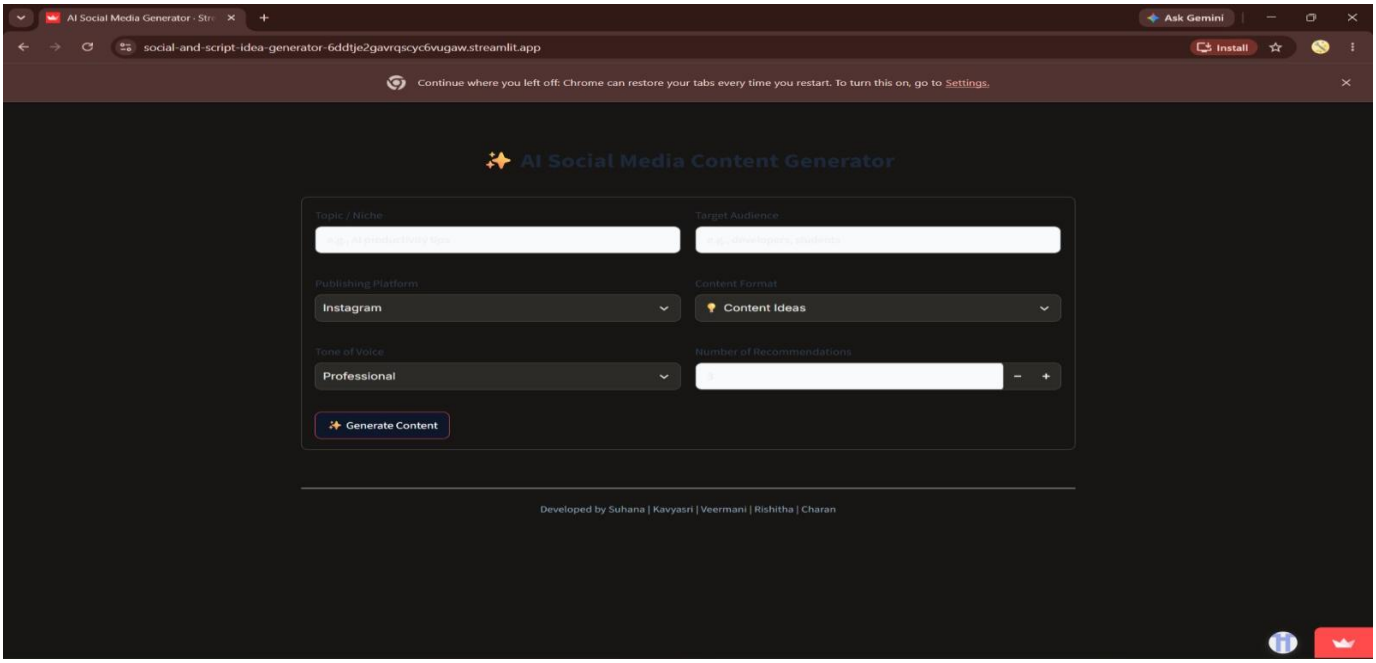


Figure 5.1.1: User Input Interface

5.1.2 Publishing Platform Selection Module

Users can select their preferred social media platform, such as Instagram, YouTube, LinkedIn, Twitter/X, Facebook, or TikTok.

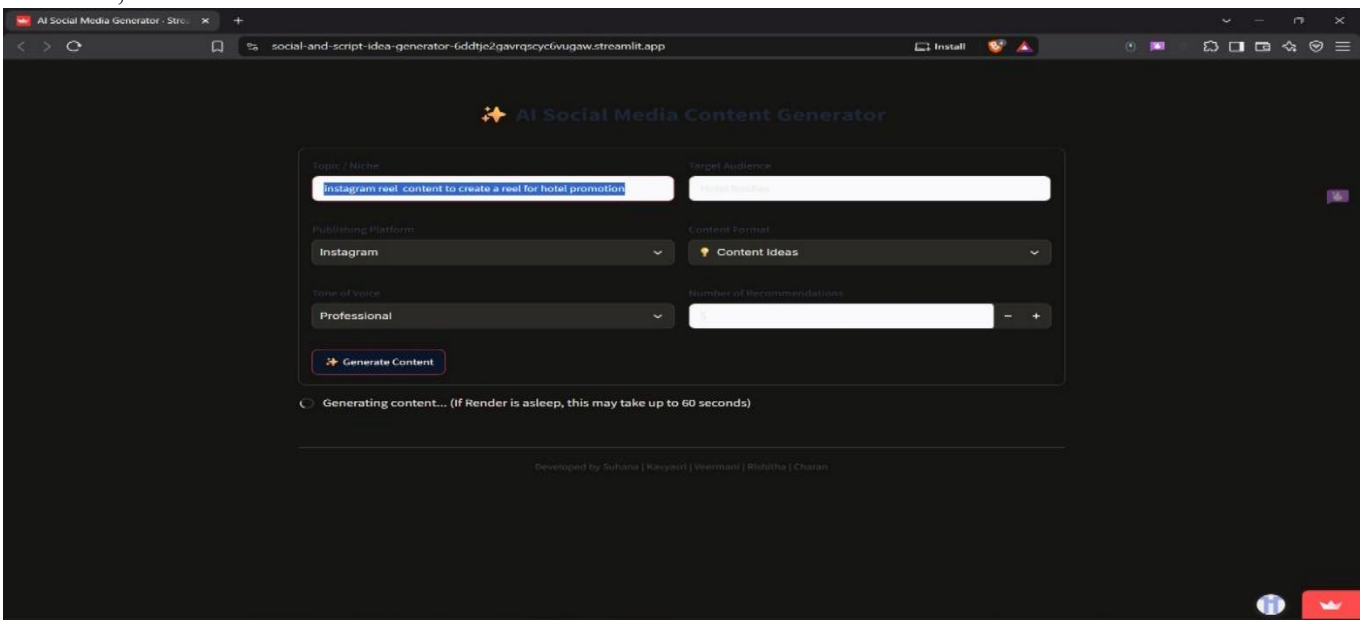


Figure 5.1.2: Publishing Platform Selection

5.1.3 Content Format and Tone Selection Module

Users can choose the required content format, such as Content Ideas, Captions + Hashtags, Full Video Script, Opening Hook, or CTA Only. They can also select a tone of voice, including Professional, Casual, Witty, Inspirational, and Educational.

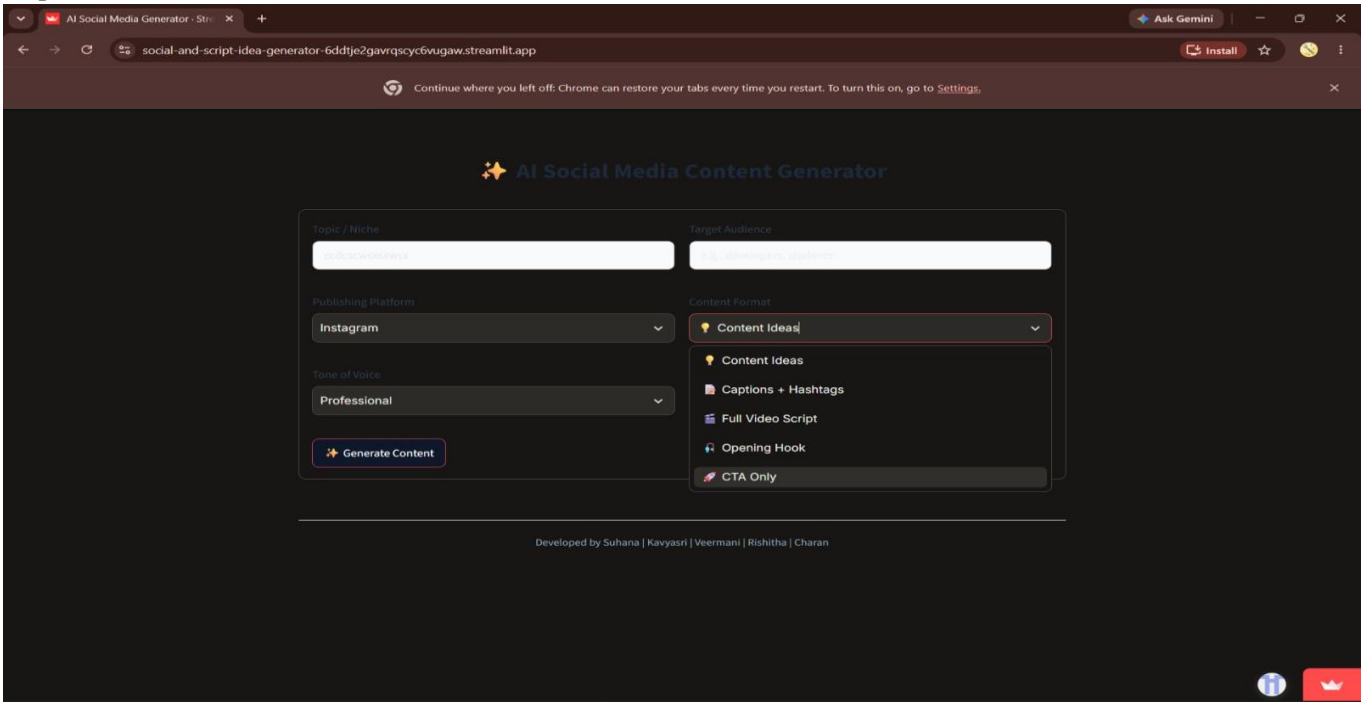


Figure 5.1.3: Content Format Selection

5.1.4 Tone of Voice Selection

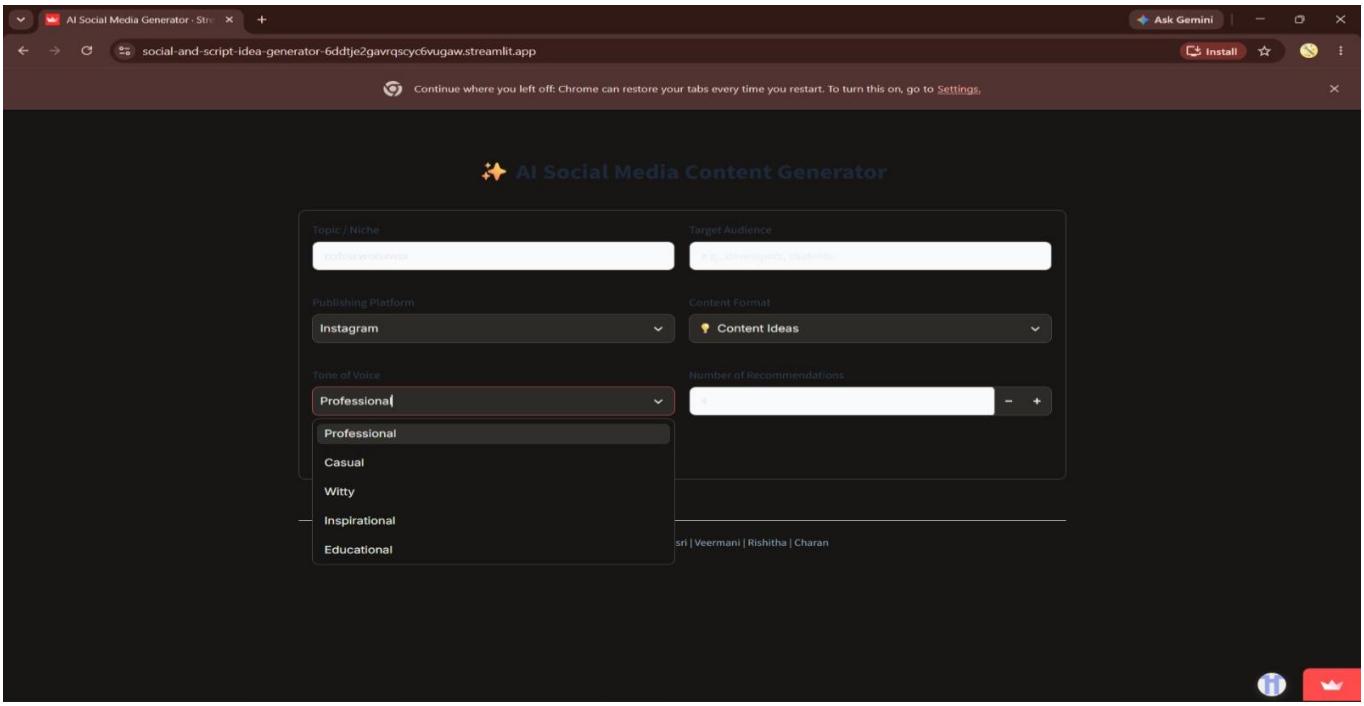


Figure 5.1.4 Tone of Voice Selection

5.1.5 AI Content Generation Module

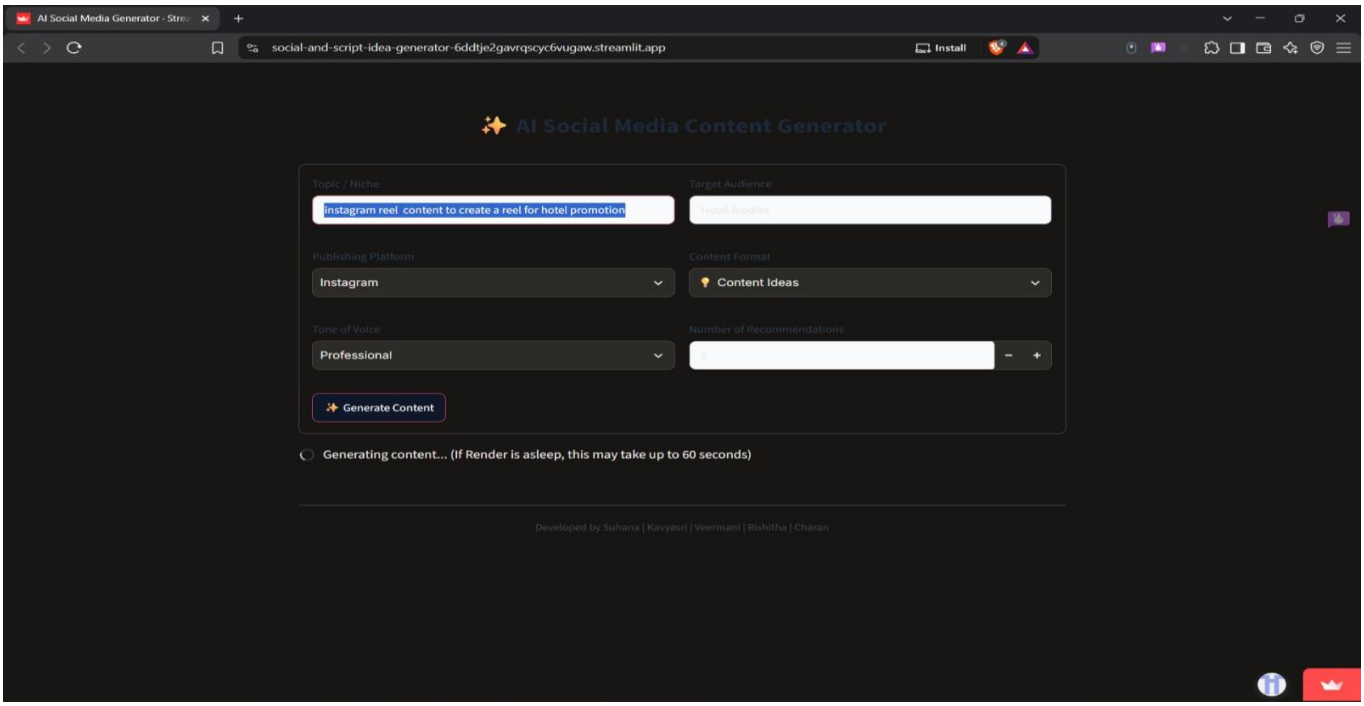


Figure 5.1.5: Content Generation Process

5.1.6 Generated Social Media Content

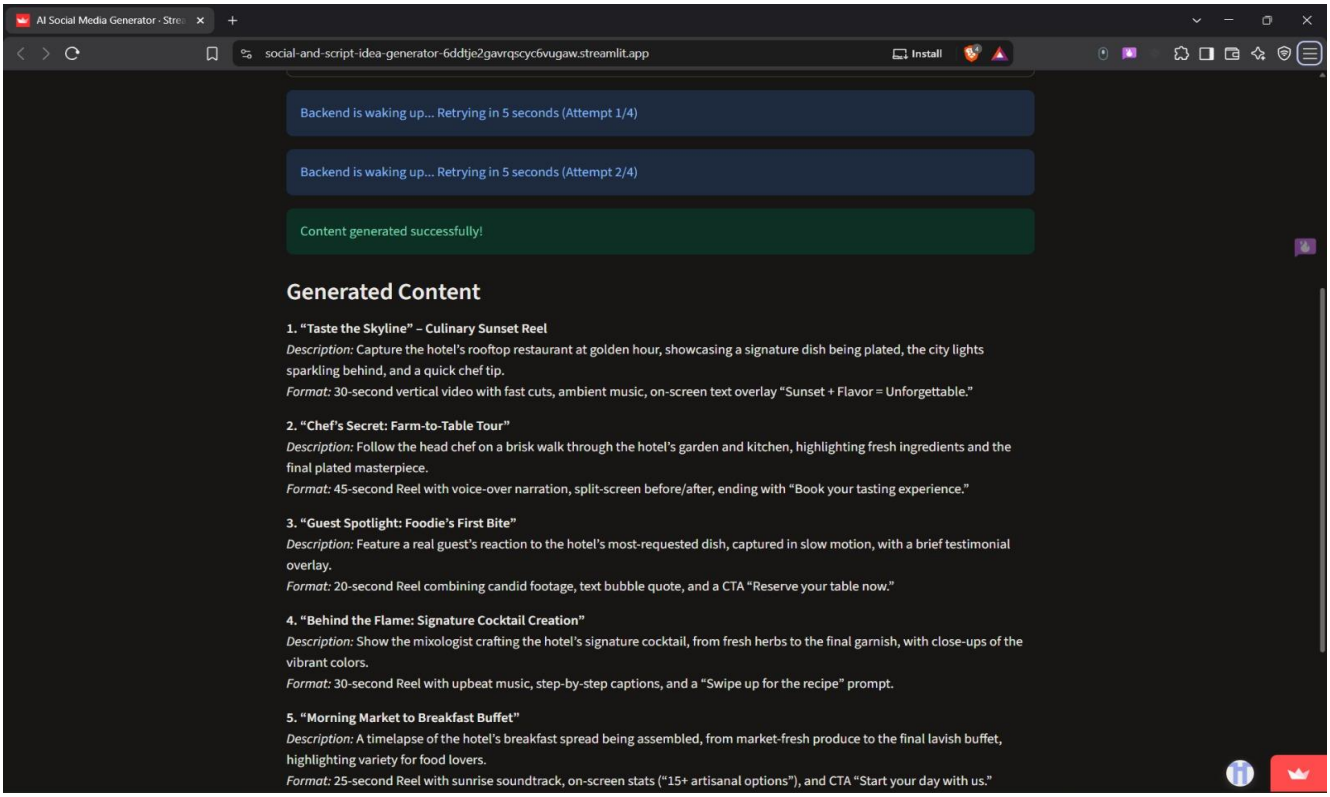


Figure 5.1.6 Generated Social Media Content

5.2 Testing Plan

Test Case	Test Procedure	Expected Result
Topic Input	Enter a topic or niche.	Topic is accepted.
Audience Input	Enter the target audience.	Audience details are accepted.
Platform Selection	Select a social media platform	Selected platform is applied
Content Format	Choose a content format.	Selected format is applied.
Tone Selection	Select a tone of voice.	Content follows the selected tone.
Number of Recommendations	Change the recommendation count	Requested number of suggestions is generated.
Content Generation	Click Generate Content	AI generates relevant content.
Output Display	View the generated results	Content is displayed successfully.
Backend Connection	Generate content when the backend is inactive.	Application retries the connection and displays the result if successful.

5.3 Technical Limitations and Considerations

- Internet Dependency: The application requires an internet connection to communicate with the Groq API and backend.
- Backend Availability: If the backend is inactive, content generation may take additional time while the service restarts.
- API Limitations: Groq API usage may be affected by rate limits, service availability, or API errors.
- Generated Content Accuracy: AI-generated suggestions may sometimes contain incorrect or irrelevant information.
- Prompt Dependency: The quality of generated content depends on the user's input and prompt instructions.
- Response Time: Content generation time may vary depending on server availability and API response speed.
- Context Limitations: The relevance of RAG-based results depends on the quality and availability of the reference information.

6. Conclusion & Future Scope

6.1 Conclusion

The AI-Powered Social Media Content and Script Idea Generator provides an easy-to-use platform for creating social media content based on user requirements. It generates content ideas, captions, hashtags, opening hooks, and video scripts according to the selected platform, target audience, and tone.

The project uses Python and Streamlit for the application interface, Groq API for AI content generation, and RAG technology to retrieve relevant information from reference documents. It helps users save time and create more relevant and engaging content.

The project demonstrates how Generative AI can simplify social media content creation and support digital marketing activities.

6.2 Future Scope

- Add support for more social media platforms and content formats.
- Provide multilingual content generation.
- Improve content quality through advanced prompt engineering.
- Support additional document formats such as Word and text files.
- Add user login and content generation history.
- Allow users to edit and download generated content.
- Integrate image and video generation features.
- Provide keyword and hashtag trend analysis.
- Improve content personalization based on brand guidelines.
- Add performance analytics to help users understand content engagement.